

Final Project - EECS 211
Winter 2025

Introduction:

Per the final project description: *“The objective of this project is to implement a dynamic tick interval policy. This should respond to the operation of processes in the system with the goal of reducing the number of ticks while preventing a process from monopolizing the CPU.”* Given the xv6-riscv-riscv.zip file, I was able to create a dynamic tick policy algorithm that adjusts the tick interval depending on whether the state of the process is running or sleeping, as well as how many processes exist. To measure the performance of my algorithm against the static tick policy, I created 3 new system calls to: retrieve the total amount of context switches that occurred during the program’s execution; retrieve the current tick rate; and retrieve the total amount of ticks during the program’s execution. From the performance analysis, I was able to significantly reduce the amount of ticks while improving the execution time in some tests.

Dynamic Tick Policy Algorithm:

I wanted my dynamic tick algorithm to increase the tick interval if fewer processes were running so that their executions may go uninterrupted from context switching and its overhead. I also wanted to increase the tick interval if a significant amount of processes were sleeping as making a constant switch to them would only incur more overhead.

```
if(proc_running > 10) {  
    dynamic_tick_rate = 200000;  
} else if(proc_running > 5) {  
    dynamic_tick_rate = 500000;  
} else if(proc_running > 2) {  
    dynamic_tick_rate = 1000000;  
} else if(proc_running > 1 || proc_sleeping > 5) {  
    dynamic_tick_rate = 2000000;  
} else if (proc_running <= 1 || proc_sleeping > 10) {  
    dynamic_tick_rate = 3000000;  
}
```

Dynamic Tick Policy Algorithm

The above snippet of code shows that the lesser amount of running processes or the greater amount of sleeping processes will increase the tick rate to 3,000,000 from the original 1,000,000. As the amount of processes increases up to 10 and beyond, the shorter the tick rate interval becomes. Conceptually, this should decrease the tick count for programs with fewer processes and make them faster as they will be seeing fewer interruptions. Also, if there are too

many processes that are sleeping, the tick interval will also increase as to not disturb the running process.

Dynamic Tick Policy Implementation:

Clockintr() gets a uint64 variable called dynamic_tick_rate that scheduler() in proc.c can write to:

```
void
clockintr()
{
    if(cpuid() == 0){
        acquire(&tickslock);
        ticks++;
        total_ticks += dynamic_tick_rate;
        wakeup(&ticks);
        release(&tickslock);
    }

    w_stimecmp(r_time() + dynamic_tick_rate);
}
```

Scheduler() gets two new variables that keep count of the number of running and sleeping processes. If the amount of running processes increases or the number of sleeping processes decreases, the tick rate will decrease:

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for(;;){

        intr_on();

        int proc_running = 0;
        int proc_sleeping = 0;

        int found = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
```

```

    acquire(&p->lock);
    if(p->state == RUNNING) {
        proc_running++;
    }
    if(p->state == SLEEPING) {
        proc_sleeping++;
    }
    if(p->state == RUNNABLE) {

        if(proc_running > 10) {
            dynamic_tick_rate = 200000;
        } else if(proc_running > 5) {
            dynamic_tick_rate = 500000;
        } else if(proc_running > 2) {
            dynamic_tick_rate = 1000000;
        } else if(proc_running > 1 || proc_sleeping > 5) {
            dynamic_tick_rate = 2000000;
        } else if (proc_running <= 1 || proc_sleeping > 10) {
            dynamic_tick_rate = 3000000;
        }

        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        total_context_switches++;
        c->proc = 0;
        found = 1;

    }
    release(&p->lock);
}

if(found == 0) {
    // nothing to run; stop running on this core until an interrupt.
    intr_on();
    asm volatile("wfi");
}
}
}

```

Performance Measurement Implementation:

Adding new sys calls for debugging and measuring performance

To measure the performance of the algorithm, I was given the `sys_uptime()` system call that can be used to calculate the amount of ticks that occurs during a program's execution. In addition to that system call, I created 3 more called `sys_get_context_switches()`, `sys_get_tick_rate()`, and `sys_get_total_ticks()`. Implementation was successful after adding the changes to `sysproc.c`, `sysproc.h`, `usys.S`, `user.h`, and `syscall.c`.

- `sys_get_context_switches()` gets the current amount of context switches. I use it to calculate the total number of context switches that occur during a programs execution
- `sys_get_tick_rate()` get the current tick rate. I use it to calculate the average tick rate and to find the execution time in seconds given a tick interval of 1,000,000 equates to 0.1sec
- `sys_get_total_ticks()` calculates the sum of the tick intervals used in calculating the execution time in seconds

`sysproc.c`

```
uint64
sys_get_context_switches(void)
{
    extern uint64 total_context_switches;
    return total_context_switches;
}

uint64
sys_get_tick_rate(void)
{
    extern uint64 dynamic_tick_rate;
    return dynamic_tick_rate;
}

uint64
sys_get_total_ticks(void)
{
    extern uint64 total_ticks;
    return total_ticks;
}
```

3 new system calls for debugging and performance measuring

sysproc.h

```
#define SYS_get_context_switches 22
#define SYS_get_tick_rate 23
#define SYS_get_total_ticks 24
```

System call numbers for the newly made sys calls

Adding print statements to determine performance of dynamic and static tick

Assuming that the CPU speed is 10MHz since the comment in trap.c states that 1,000,000 equates to 0.1sec, I was able to calculate the execution time for the individual program tests. When running the entire usertests suite, I could not return a valid execution time, so it was done manually.

```
int start = uptime();
int context_start = get_context_switches();
int tick_avg_start = get_total_ticks();

// run the test here

int end = uptime();
int elapsed_time = (end - start);
int dynamic_tick_rate = get_tick_rate();
int context_end = get_context_switches();
printf("\nExecution Time: %d ticks\n", elapsed_time);
printf("Execution Time: %d.%d seconds\n",
(elapsed_time*dynamic_tick_rate)/10000000, ((elapsed_time*dynamic_tick_rate
)%10000000)/(10000000/10));
printf("Total Context Switches: %d \n", context_end - context_start);
int tick_avg_end = get_total_ticks();
int tick_avg = (tick_avg_end - tick_avg_start) / elapsed_time;
printf("Tick AVG: %d \n", tick_avg);
```

ALL TESTS PASSED

```
Execution Time: 184 ticks
Execution Time: 55.2 seconds
Total Context Switches: 394
Tick AVG: 3000000
$
```

Example usage of the print statements

Results and Discussion:

Running 'ls'								
	Static Tick Policy				Dynamic Tick Policy			
Metric:	Exec Time (sec):	Total Ticks:	Total Context Switches:	Avg Context Switch Interval:	Exec Time (sec):	Total Ticks:	Total Context Switches:	Avg Context Switch Interval:
Run 1:	0.5	5	5	1000000	0.3	1	1	3000000
Run 2:	0.5	5	5	1000000	0.6	2	2	3000000
Run 3:	0.4	4	4	1000000	0.6	2	2	3000000
Run 4:	0.4	4	4	1000000	0.3	1	1	3000000
Run 5:	0.5	5	5	1000000	0.3	1	1	3000000
Run 6:	0.4	4	4	1000000	0.3	1	1	3000000
Run 7:	0.4	4	4	1000000	0.6	2	2	3000000
Run 8:	0.6	6	6	1000000	0.3	1	1	3000000
Run 9:	0.4	4	4	1000000	0.3	1	1	3000000
Run 10:	0.4	4	4	1000000	0.3	1	1	3000000
AVG:	0.45	4.5	4.5	1000000	0.39	1.3	1.3	3000000
MAX:	0.6	6	6	1000000	0.6	2	2	3000000
MIN:	0.4	4	4	1000000	0.3	1	1	3000000

Discussion of running ls:

I was able to see a minor improvement in execution time as the average execution time lessened from 0.45 to 0.39 seconds. The total amount of ticks decreases dramatically from an average of 4.5 to 1.3 ticks. Fewer context switches occurred with dynamic ticks as well.

Running 'forktest'								
	Static Tick Policy				Dynamic Tick Policy			
Metric:	Exec Time (sec):	Total Ticks:	Total Context Switches:	Avg Context Switch Interval:	Exec Time (sec):	Total Ticks:	Total Context Switches:	Avg Context Switch Interval:
Run 1:	0.1	1	63	1000000	0.3	1	63	3000000
Run 2:	0.2	2	63	1000000	0.3	2	62	1500000
Run 3:	0.2	2	63	1000000	0.3	1	63	3000000
Run 4:	0.1	1	62	1000000	0.3	1	64	3000000
Run 5:	0.2	2	63	1000000	0.3	1	67	3000000
Run 6:	0.2	2	63	1000000	0.3	1	63	3000000
Run 7:	0.2	2	63	1000000	0.3	1	63	3000000
Run 8:	0.1	1	64	1000000	0.6	2	63	3000000
Run 9:	0.2	2	63	1000000	0.3	1	64	3000000
Run 10:	0.2	2	63	1000000	0.3	1	63	3000000
AVG:	0.17	1.7	63	1000000	0.33	1.2	63.5	2850000
MAX:	0.2	2	64	1000000	0.6	2	67	3000000
MIN:	0.1	1	62	1000000	0.3	1	62	1500000

Discussion of running forktests:

Observed slightly worse execution time with dynamic ticks, with a small reduction in total ticks. There is almost no difference between the total amount of context switches.

Running all 'usertests'						
	Static Tick Policy			Dynamic Tick Policy		
Metric:	Exec Time (sec):	Total Ticks:	Total Context Switches:	Exec Time (sec):	Total Ticks:	Total Context Switches:
Run 1:	970	9706	97157	1110	3826	92898
Run 2:	1167	11673	102718	1220	4331	97998
Run 3:	979	9792	98671	1192	4335	96491
Run 4:	1135	11358	99477	1120	4072	95776
Run 5:	1189	11895	99025	1200	4289	97449
AVG:	1121.1	11216.7	99757.7	1168.4	4170.6	96122.4
MAX:	1233	12339	102718	1220	4335	97998
MIN:	970	9706	97157	1110	3826	92898

Discussion of running usertests:

Execution time between static tick and dynamic tick policy is nearly the same. I observed total ticks being reduced to a third of the original.

Conclusion:

From the results above I was able to see a significant decrease in the amount of ticks after the change to a dynamic tick policy. Depending on the program being executed, the total amount of context switches also decreased. However, the average execution time between static and dynamic tick policies is relatively comparable. Ideally, I would expect a significant decrease in execution time, but saw contrary in some cases such as with forkttest. This may be due to the delay from the if statements being checked when the process enters the running state in proc.c.